Appunti di Calcolo Numerico

16 marzo 2026

Indice

1 Introduzione **2**

2 Teoria dellla computabilità **3**

2.1 Fondamenti della Calcolabilità . . . 3

2.1.1 Enumerabilità, computabilità e decidibilità . . . 3

2.2 Funzioni Ricorsive . . . 6

2.2.1 Funzioni ricorsive primitive . . . 6

2.2.2 Relazioni ricorsive primitive . . . 11

2.2.3 Funzione di Ackermann e funzioni μ-ricorsive . . . 14

2.2.4 Funzioni μ-ricorsive . . . 16

2.3 Macchine di Turing . . . 17

2.3.1 Macchine di Turing come accettatori di linguaggi . . . 21

2.3.2 Varianti di Macchine di Turing . . . 23

2.4 MDT Non Deterministica . . . 26

2.5 Funzioni Parziali Computabili . . . 27

1 Introduzione

L'Informatica Teorica si divide in due macro-aree fondamentali:

- **Teoria della computabilità**: cerca modelli formali che si avvicinino al concetto logico e matematico di ALGORITMO.
- **Teoria della complessità computazionale**: si occupa di determinare la quantità minima di risorse (principalmente spazio e tempo) necessarie ad un algoritmo per risolvere un dato problema.

2 Teoria dellla computabilità

2.1 Fondamenti della Calcolabilità

2.1.1 Enumerabilità, computabilità e decidibilità

Cos'è un algoritmo? È una NOZIONE PRIMITIVA $\longrightarrow$ non ha una definizione formale, ma lo abbiamo nella nostra testa e sappiamo riconoscerlo.

Definizioni formali
Per prima cosa ci mettiamo in un <u>contesto formale</u>:

- Σ : alfabeto
- Σ^* : stringhe finite su Σ
- $\mathcal{L} \subseteq \Sigma^*$: linguaggio

Definizione: $\mathcal{L} \subseteq \Sigma^*$ si dice **DECIDIBILE** quando $\exists\ \mathcal{M}$ algoritmo che, data $w \in \Sigma^*$, mi dice in tempo finito se $w \in \mathcal{L}$ oppure $w \notin \mathcal{L}$.

Definizione: $\mathcal{L} \subseteq \Sigma^*$ si dice **ENUMERABILE** quando $\exists\ \mathcal{M}$ algoritmo che, fatto partire su input vuoto, <u>produce la lista delle stringhe di $\mathcal{L}$</u> (ciascuna stringa in tempo finito).

Definizione: Siano Σ_1, Σ_2 alfabeti, $\mathcal{L}_1 \subseteq \Sigma_1^*$, $\mathcal{L}_2 \subseteq \Sigma_2^*$.
$f : \mathcal{L}_1 \to \mathcal{L}_2$ si dice **COMPUTABILE** quando $\exists\ \mathcal{M}$ algoritmo che, fatto partire su input $w \in \mathcal{L}_1$, restituisce $f(w) \in \mathcal{L}_2$ in tempo finito.

Proposizione: Sia Σ un alfabeto.

$$\mathcal{L} \text{ DECIDIBILE} \implies \mathcal{L} \text{ ENUMERABILE}$$

Dimostrazione:
Sia $\mathcal{M}$ un algoritmo di decisione per $\mathcal{L}$. Descrivo un algoritmo di enumerazione per $\mathcal{L}$: $\forall$ stringa $w \in \Sigma^*$, eseguo $\mathcal{M}$ su w:

- Se $w \in \mathcal{L}$, aggiungo w alla lista.
- Se $w \notin \mathcal{L}$, non l'aggiungo.

Osservazione: Σ numerabile $\implies \Sigma^*$ enumerabile.
Sia $\Sigma = \{a_1, a_2, \dots, a_n, \dots\}$.
Se cerchiamo di enumerare le stringhe di lunghezza 2 su Σ tramite un PROCEDIMENTO DIAGONALE DI CANTOR:

$$\begin{array}{c|ccccccc}
 & a_1 & & a_2 & & a_3 & & \dots \\
\hline
a_1 & a_1a_1 & \rightarrow & a_1a_2 & & a_1a_3 & \rightarrow & \dots \\
 & & \swarrow & & \nearrow & & \swarrow & \\
a_2 & a_2a_1 & & a_2a_2 & & a_2a_3 & & \dots \\
 & \downarrow & \nearrow & & \swarrow & & & \\
a_3 & a_3a_1 & & a_3a_2 & & \dots & & \\
 & & \swarrow & & & & & \\
\vdots & \vdots & & \vdots & & & & \ddots
\end{array}$$

Usando il procedimento diagonale è possibile enumerare le parole e poi trovare un algoritmo di enumerazione per le stringhe.

Attenzione: $\mathcal{L}$ decidibile $\impliedby$ $\mathcal{L}$ enumerabile (NON vale il viceversa in generale).
MA vale la seguente...

Proposizione:

$$\mathcal{L} \text{ DECIDIBILE} \iff \mathcal{L} \text{ ENUMERABILE e } \mathcal{L}^c \text{ ENUMERABILE} \quad (\text{dove } \mathcal{L}^c = \Sigma^* \setminus \mathcal{L})$$

Dimostrazione:

$\Longrightarrow$) $\mathcal{L}$ decidibile $\Longrightarrow$ $\mathcal{L}$ enumerabile (già dimostrato, OK).
$\mathcal{L}$ decidibile $\Longrightarrow$ $\mathcal{L}^c$ enumerabile: analogo.

$\Longleftarrow$) Sia $\mathcal{M}$ algoritmo di enumerazione per $\mathcal{L}$: $w_1, w_2, w_3, \ldots, w_n, \ldots$
Sia $\mathcal{M}^c$ algoritmo di enumerazione per $\mathcal{L}^c$: $v_1, v_2, v_3, \ldots, v_n, \ldots$
Costruisco un algoritmo di decisione per $\mathcal{L}$:
Su input $w \in \Sigma^*$, eseguo $\mathcal{M}$ e $\mathcal{M}^c$ in maniera ALTERNATA. Ogni volta che viene prodotta una stringa X, la confronto con w:

- Se $X \neq w$, proseguo.
- Se $X = w$ e X è prodotta da $\mathcal{M}$, accetto w.
- Se $X = w$ e X è prodotta da $\mathcal{M}^c$, rifiuto w.

Definizione: $\mathcal{L} \subseteq \Sigma^*$ si dice **SEMIDECIDIBILE** quando $\exists$ $\mathcal{M}$ algoritmo che, fatto partire su input w:

- Se $w \in \mathcal{L}$, termina e accetta w.
- Se $w \notin \mathcal{L}$, può non terminare (va in loop).

Osservazione: $\mathcal{L}$ semidecidibile $\iff$ $\exists$ $\mathcal{M}$ algoritmo che, su input w:

- Se $w \in \mathcal{L}$, termina e accetta w.
- Se $w \notin \mathcal{L}$, non termina.

Proposizione:

$$\mathcal{L} \text{ ENUMERABILE} \iff \mathcal{L} \text{ SEMIDECIDIBILE}$$

Dimostrazione:

$\Longrightarrow$) Sia $\mathcal{M}$ algoritmo di enumerazione per $\mathcal{L}$: $w_1, w_2, w_3, \ldots$
Costruisco un algoritmo di semidecisione per $\mathcal{L}$:
Data $w \in \Sigma^*$, confronto w con le stringhe prodotte via via da $\mathcal{M}$. Se w è tra queste, termino e accetto w.

$\Longleftarrow$) Sia $\mathcal{M}$ algoritmo di semidecisione per $\mathcal{L}$ (che non termina su tutte le stringhe del complemento).
Eseguo i passi di computazione intrecciandoli (dovetailing):

$$\begin{array}{c|cccccccc}
w_1: & x & & x & & x & \ldots & x \\
 & & \swarrow & & \nearrow & & \swarrow & \\
w_2: & x & & x & & x & \ldots & \circ \\
 & \downarrow & \nearrow & & \swarrow & & & \\
w_3: & x & & x & & \ldots & & \\
 & & \swarrow & & & & & \\
w_4: & x & & \ldots & & & & \\
\vdots & \vdots & & & & & &
\end{array}$$

(Eseguo 1 passo su w_1; poi 1 passo su w_1 e 1 su w_2; poi 1 passo su w_1, w_2 e w_3, ecc. Quando una computazione termina e accetta (es. in w_2), metto la stringa corrispondente nella lista).

Definizione: $\mathcal{L} \subseteq \Sigma^*$ si dice avere **FUNZIONE CARATTERISTICA** $\chi_{\mathcal{L}}$ se la funzione è così definita:

$$\chi_{\mathcal{L}} : \Sigma^* \to \{0,1\} \quad : \quad w \mapsto \begin{cases} 0, & w \notin \mathcal{L} \\ 1, & w \in \mathcal{L} \end{cases}$$

Proposizione:

$$\mathcal{L} \text{ DECIDIBILE} \iff \chi_{\mathcal{L}} \text{ COMPUTABILE}$$

Dimostrazione:

$\Longrightarrow$) Sia $\mathcal{M}$ algoritmo di decisione per $\mathcal{L}$. Costruisco un algoritmo di calcolo per $\chi_{\mathcal{L}}$:
Data $w \in \Sigma^*$, eseguo $\mathcal{M}$ su w:

- Se $w \in \mathcal{L}$, scrivo 1.
- Se $w \notin \mathcal{L}$, scrivo 0.

$\Longleftarrow$) Sia $\mathcal{M}$ algoritmo di calcolo per $\chi_{\mathcal{L}}$. Costruisco un algoritmo di decisione per $\mathcal{L}$:
Su input w, calcolo $\chi_{\mathcal{L}}(w)$ (usando $\mathcal{M}$):

- Se $\chi_{\mathcal{L}}(w) = 1$, $w \in \mathcal{L}$.
- Se $\chi_{\mathcal{L}}(w) = 0$, $w \notin \mathcal{L}$.

Proposizione: Siano $\mathcal{L}_1, \mathcal{L}_2 \subseteq \Sigma^*$ DECIDIBILI. Allora:

1. $\mathcal{L}_1 \cup \mathcal{L}_2$ è decidibile
2. $\mathcal{L}_1 \cap \mathcal{L}_2$ è decidibile
3. $\mathcal{L}_1^c$ è decidibile

Dimostrazione:

1. **Algoritmo di decisione per $\mathcal{L}_1 \cup \mathcal{L}_2$:**
   Su input w: Eseguo $\mathcal{M}_1$. Se $w \in \mathcal{L}_1$ termino e accetto.
   Se $w \notin \mathcal{L}_1$, eseguo $\mathcal{M}_2$:
   - Se $w \in \mathcal{L}_2$, termino e accetto.
   - Altrimenti rifiuto.
2. **Algoritmo di decisione per $\mathcal{L}_1 \cap \mathcal{L}_2$:**
   Su input w: Eseguo $\mathcal{M}_1$:
   - Se $w \notin \mathcal{L}_1$, rifiuto.
   - Se $w \in \mathcal{L}_1$, osservo $\mathcal{M}_2$:
     - Se $w \notin \mathcal{L}_2$, rifiuto.
     - Altrimenti accetto.
3. **Algoritmo di decisione per $\mathcal{L}_1^c$:**
   Su input w, eseguo $\mathcal{M}_1$ e SCAMBIO LE SUE RISPOSTE (cioè l'algoritmo di decisione inverte l'output).

2.2 Funzioni Ricorsive

2.2.1 Funzioni ricorsive primitive

Ci concentriamo su funzioni del tipo:

$$f : \mathbb{N}^k \to \mathbb{N}$$
$$(x_1, \dots, x_k) \in \mathbb{N}^k$$
$$f(x_1, \dots, x_k)$$

Indicheremo il vettore di input come $\vec{x} = (x_1, \dots, x_k)$.

FUNZIONI INIZIALI

1. **Costanti k-arie di valore 0**:
$$C_0^{(k)} : \mathbb{N}^k \to \mathbb{N}$$
$$C_0^{(k)}(\vec{x}) = 0$$

2. **Funzione successore**:
$$S : \mathbb{N} \to \mathbb{N}$$
$$S(x) = x + 1$$

3. **Proiezioni j-esime k-arie**:
$$\mathcal{E}_j^{(k)} : \mathbb{N}^k \to \mathbb{N}$$
$$\mathcal{E}_j^{(k)}(\vec{x}) = x_j \quad \text{(estrae l'argomento j-esimo)}$$

Definizione: Sia A un insieme, Σ un alfabeto.
Una **Codifica** è una funzione $p : A \to \Sigma^*$ (da A ottengo una stringa).

Definizione: Una **Rappresentazione** (Buona codifica) di A in Σ si ha quando:

1. p è **computabile** ($\exists$ algoritmo che per $\forall\ a \in A$ mi calcola $p(a)$).
2. p è **iniettiva** (la codifica è unica).
3. $p(A)$ è **decidibile** e p è **effettivamente invertibile**, cioè: $\exists$ un algoritmo che, $\forall\ w \in p(A)$, mi restituisce $a \in A$ tale che $p(a) = w$ (Posso ricostruire senza ambiguità l'elemento di A dalla codifica).

Proposizione: Sia Σ un alfabeto. Allora $\exists\ p$ rappresentazione in $\{0,1\}^*$.
$p : \Sigma^* \to \{0,1\}^*$ (Posso codificare in binario).
Dimostrazione:
Sia $\Sigma = \{a_1, a_2, \dots, a_n, \dots\}$.
$p : \Sigma^* \to \{0,1\}^*$
$p(a_{i_1} a_{i_2} \dots a_{i_n}) = 01\dots101\dots1\dots01\dots1$ (dove i blocchi di 1 dipendono dagli indici i_k).
p è una rappresentazione di Σ^* in $\{0,1\}^*$.

Codifica unaria per numeri naturali:
$\Sigma = \{1\}$
$0 \mapsto 1$
$1 \mapsto 11$
$2 \mapsto 111$
$n \mapsto \underbrace{11\dots1}_{n+1 \text{ volte}}$

Osservazione: Le funzioni iniziali sono computabili.
Esempio: $S(\underbrace{11\dots1}_{n+1}) = \underbrace{1\dots1}_{n+2}$

Definizione: Composizione generalizzata
Date $g_1 \dots g_n : \mathbb{N}^k \to \mathbb{N}$ e $f : \mathbb{N}^n \to \mathbb{N}$.
La **composizione generalizzata** (o sostituzione) di f con $(g_1, \dots, g_n)$ è la funzione:

$$f \circ (g_1, \dots, g_n) : \mathbb{N}^k \to \mathbb{N}$$

$$(f \circ (g_1, \dots, g_n))(\vec{x}) = f(g_1(\vec{x}), \dots, g_n(\vec{x}))$$

Proposizione: $f, g_1, \dots, g_n$ computabili $\implies f \circ (g_1, \dots, g_n)$ computabile.
Dimostrazione:
Siano $F, G_1, \dots, G_n$ algoritmo di calcolo per $f, g_1, \dots, g_n$.
Algoritmo di calcolo per $f \circ (g_1, \dots, g_n)$:
Data $\vec{x} \in \mathbb{N}^k$:

- Uso $G_1, \dots, G_n$ per calcolare $g_1(\vec{x}), \dots, g_n(\vec{x})$.
- Uso F per calcolare $f(g_1(\vec{x}), \dots, g_n(\vec{x}))$.

Definizione: Ricorsione Primitiva (RP)
Date $g : \mathbb{N}^k \to \mathbb{N}$, $\varphi : \mathbb{N}^{k+2} \to \mathbb{N}$.
Diciamo che $f : \mathbb{N}^{k+1} \to \mathbb{N}$ è definita per **ricorsione primitiva** (RP) da g e φ quando:

- $f(\vec{x}, 0) = g(\vec{x})$ *(caso base)*
- $f(\vec{x}, y+1) = \varphi(\vec{x}, y, f(\vec{x}, y))$ *(passo induttivo)*

Proposizione: g, φ computabili $\implies f$ definita per RP da g, φ è computabile.
Dimostrazione:
Descrivo un algoritmo per calcolare f, ragionando per induzione sull'ultimo parametro di f. In pratica facciamo vedere che fissato $\vec{x} \in \mathbb{N}^k$, per ogni $y \in \mathbb{N}$, so calcolare $f(\vec{x}, y)$:

- (Base $y = 0$): $f(\vec{x}, 0) = g(\vec{x})$, e g è computabile.
- (Iterazione): Supponiamo di saper calcolare $f(\vec{x}, y)$.
  $f(\vec{x}, y+1) = \varphi(\vec{x}, y, f(\vec{x}, y))$. Essendo φ computabile (per hp) e $f(\vec{x}, y)$ noto, anche il calcolo al passo successivo restituisce un risultato in tempo finito.

Definizione: Derivazione Ricorsiva Primitiva (drp)
È una sequenza di funzioni $f_1, f_2, \dots, f_n$ t.c. $\forall\ i = 1, \dots, n$:

- f_i è **funzione iniziale**, oppure
- f_i è **ottenuta da** f_j, f_k **per RP** con $j, k < i$, oppure
- f_i è **ottenuta per composizione generalizzata** da funzioni precedenti

Definizione: Funzione Ricorsiva Primitiva (rp)
Una **funzione ricorsiva primitiva (rp)** è una funzione che compare in coda ad una drp (ovvero è l'ultima funzione di una drp).

Osservazione:

1. Ogni prefisso di una drp è una drp.
2. Ogni funzione che compare in una drp è rp.

Proposizione: Ogni funzione rp è computabile.
Dimostrazione:
Sia f rp. Dimostro per **induzione sulla lunghezza n di una drp** che termina con f:

- **Caso base** ($n = 1$): $f \implies f$ è funzione iniziale, dunque è computabile.
- **Passo induttivo**: Supponiamo che tutte le funzione che compaiono in una drp di lunghezza $< n$ siano computabili.
  Sia $f_1, \ldots, f_n = f$.
  f deriva da funzioni precedenti $f_j, \ldots, f_k$ le quali sono computabili per Hp induttiva. Siccome le operazioni di composizione e RP mantengono la computabilità (come dimostrato in precedenza), anche f è computabile.

Proposizione:

1. $g : \mathbb{N}^k \to \mathbb{N}$, $\varphi : \mathbb{N}^{k+2} \to \mathbb{N}$ rp $\implies f$ ottenuta per RP da g e φ è rp.
2. $\varphi : \mathbb{N}^n \to \mathbb{N}$, $g_1, \ldots, g_n : \mathbb{N}^k \to \mathbb{N}$ rp $\implies f \circ (g_1, \ldots, g_n)$ rp.

Dimostrazione:

1. Siano $g_1, \ldots, g_s, g$ una drp per g, e $\varphi_1, \ldots, \varphi_r, \varphi$ una drp per φ.
   La sequenza $g_1, \ldots, g_s, g, \varphi_1, \ldots, \varphi_r, \varphi, f$ è essa stessa una drp (poiché per ipotesi tutte le precedenti sono drp e f è ottenuta da esse). Di conseguenza, f è rp perché compare in una drp.

ESEMPI DI FUNZIONI RICORSIVE PRIMITIVE

1) Costante m-esima k-aria: $C_m^{(k)} : \mathbb{N}^k \to \mathbb{N}$, dove $C_m^{(k)}(x_1, \ldots, x_k) = m \quad (m \in \mathbb{N})$, con k che indica il numero di argomenti (es. $C_0^{(2)}(x, y) = 0$) e m che indica il valore della costante (es. $C_3^{(2)}(x, y) = 3$). Dimostriamo per induzione su $m \in \mathbb{N}$ che è rp:

- Base: $C_0^{(k)}$ è rp (perché è una **funzione iniziale**).
- Supponiamo che $C_m^{(k)}$ sia rp.
  $C_{m+1}^{(k)}(\vec{x}) = m + 1 = (S \circ C_m^{(k)})(\vec{x})$
  Essendo la composizione tra S (funzione successore, che è iniziale) e $C_m^{(k)}$ (rp per ipotesi induttiva), allora $C_{m+1}^{(k)}$ è rp.

2) Funzione Predecessore: $v : \mathbb{N} \to \mathbb{N}$

$$v(x) = \begin{cases} x - 1, & x > 0 \\ 0, & x = 0 \end{cases}$$

Costruiamo la ricorsione primitiva:

$$\begin{cases} v(0) = 0 \quad \text{(che corrisponde a } C_0^{(0)} \text{ che è rp)} \\ v(x+1) = \left(\mathcal{E}_1^{(2)}\right)(x, v(x)) = x \quad \text{(rp perché } \mathcal{E}_1^{(2)} \text{ è funz. iniziale)} \end{cases}$$

$\implies v(x)$ è rp.

3) Somma: $s : \mathbb{N}^2 \to \mathbb{N}$
$s(x, y) = x + y$

$$\begin{cases} s(x,0) = x = \mathcal{E}_1^{(1)}(x) \quad \text{(rp)} \\ s(x, y+1) = x + y + 1 = \left(S \circ \mathcal{E}_3^{(3)}\right)(x, y, s(x,y)) \quad \text{(rp)} \end{cases}$$

$\implies$ s è rp.

4) Prodotto: $p : \mathbb{N}^2 \to \mathbb{N}$
$p(x, y) = x \cdot y$

$$\begin{cases} p(x,0) = 0 = \left(C_0^{(1)}\right)(x) \quad \text{(rp)} \\ p(x, y+1) = xy + x = \left(s \circ (\mathcal{E}_3^{(3)}, \mathcal{E}_1^{(3)})\right)(x, y, p(x,y)) \quad \text{(rp)} \end{cases}$$

$\implies$ p è rp.

5) Esponenziazione: $\pi : \mathbb{N}^2 \to \mathbb{N}$
$\pi(x, y) = x^y$

$$\begin{cases} \pi(x,0) = x^0 = 1 = C_1^{(1)}(x) \quad \text{(rp)} \\ \pi(x, y+1) = x^y \cdot x = \left(p \circ (\mathcal{E}_3^{(3)}, \mathcal{E}_1^{(3)})\right)(x, y, \pi(x,y)) \quad \text{(rp)} \end{cases}$$

6) Fattoriale: $f : \mathbb{N} \to \mathbb{N}$
$f(x) = x!$

$$\begin{cases} f(0) = 1 \quad \text{(rp)} \\ f(x+1) = (x+1)! = (x+1) \cdot x! = \left(p \circ (S \circ \mathcal{E}_1^{(2)}, \mathcal{E}_2^{(2)})\right)(x, f(x)) \end{cases}$$

7) Differenza Troncata: $\dot{-} : \mathbb{N}^2 \to \mathbb{N}$

$$x \dot{-} y = \begin{cases} x - y, & x \geq y \\ 0, & x < y \end{cases}$$

$$\begin{cases} x \dot{-} 0 = x = \mathcal{E}_1^{(1)}(x) \quad \text{(rp)} \\ x \dot{-} (y+1) = x - y - 1 = \left(v \circ \mathcal{E}_3^{(3)}\right)(x, y, x \dot{-} y) \quad \text{(rp)} \end{cases}$$

8) Distanza: $d : \mathbb{N}^2 \to \mathbb{N}$
$d(x, y) = |x - y|$

$$d(x,y) = (x \dot{-} y) + (y \dot{-} x)$$

(Questo copre entrambi i casi in cui $x \geq y$ o $y > x$)

$$= \left(s \circ (\dot{-} \circ (\mathcal{E}_1^{(2)}, \mathcal{E}_2^{(2)}), \dot{-} \circ (\mathcal{E}_2^{(2)}, \mathcal{E}_1^{(2)}))\right)(x, y)$$

$\implies$ d è rp.

9) Segno: $sg : \mathbb{N} \to \mathbb{N}$

$$sg(x) = \begin{cases} 1, & x > 0 \\ 0, & x = 0 \end{cases}$$

$$\begin{cases} sg(0) = 0 \\ sg(x+1) = 1 = C_1^{(2)}(x, sg(x)) \end{cases}$$

10) Segno Opposto: $\overline{sg} : \mathbb{N} \to \mathbb{N}$

$$\overline{sg}(x) = \begin{cases} 0, & x > 0 \\ 1, & x = 0 \end{cases}$$

Dimostrazione: Possiamo dimostrare che è rp per ricorsione primitiva:

$$\begin{cases} \overline{sg}(0) = 1 = C_1^{(0)} \quad \text{(rp)} \\ \overline{sg}(x+1) = 0 = C_0^{(2)}(x, \overline{sg}(x)) \quad \text{(rp, con composizione di proiezioni se necessario)} \end{cases}$$

$\implies \overline{sg}$ è rp.

Metodo alternativo (per composizione): Possiamo sfruttare la differenza troncata. Sottrarre x a 1 ci restituisce 1 solo se $x = 0$, altrimenti ci restituisce 0:

$$\overline{sg}(x) = 1 \dot{-} x = \left(\dot{-} \circ (C_1^{(1)}, \mathcal{E}_1^{(1)})\right)(x)$$

Essendo ottenuta per composizione di funzioni rp, è rp.

11) Delta di Kronecker: $\delta(x, y)$

$$\delta(x, y) = \begin{cases} 1, & x = y \\ 0, & x \neq y \end{cases}$$

Dimostrazione: Sappiamo che la funzione distanza $d(x, y) = |x - y|$ (che abbiamo già dimostrato essere rp) vale 0 se e solo se $x = y$, ed è strettamente maggiore di 0 altrimenti. Se applichiamo la funzione *Segno Opposto* al risultato della distanza, otteniamo esattamente il comportamento del Delta di Kronecker:

$$\delta(x, y) = \overline{sg}(d(x, y)) = (\overline{sg} \circ d)(x, y)$$

Essendo il Delta di Kronecker ottenuto per composizione di due funzioni rp ($\overline{sg}$ e d), è esso stesso una funzione rp.

Definizione: Operatore di Somma Limitata
Sia $f : \mathbb{N}^{k+1} \to \mathbb{N}$. Definiamo $g : \mathbb{N}^{k+1} \to \mathbb{N}$ ponendo:

$$g(\vec{x}, z) = \sum_{y=0}^{z} f(\vec{x}, y)$$

Diciamo che g è ottenuta per **somma limitata** da f. La funzione che, data f, costruisce g si chiama operatore di somma limitata.

Proposizione: f rp $\implies g$ ottenuta per somma limitata da f è rp.
Dimostrazione:

$$\begin{cases} g(\vec{x}, 0) = \sum_{y=0}^{0} f(\vec{x}, y) = f(\vec{x}, 0) = \left(f \circ (\mathcal{E}_1^{(k)}, \mathcal{E}_2^{(k)}, \ldots, \mathcal{E}_k^{(k)}, C_0^{(k)})\right)(\vec{x}) \\ \\ g(\vec{x}, z+1) = \sum_{y=0}^{z+1} f(\vec{x}, y) = \sum_{y=0}^{z} f(\vec{x}, y) + f(\vec{x}, z+1) = \\ = \left(s \circ (\mathcal{E}_{k+2}^{(k+2)}, f \circ (\mathcal{E}_1^{(k+2)}, \ldots, \mathcal{E}_k^{(k+2)}, S \circ \mathcal{E}_{k+1}^{(k+2)}))\right)(\vec{x}, z, g(\vec{x}, z)) \end{cases}$$

Osservazione:
Data $f : \mathbb{N}^{k+2} \to \mathbb{N}$, possiamo avere $g : \mathbb{N}^{k+1} \to \mathbb{N}$ definita come:

$$g(\vec{x}, z) = \sum_{y=0}^{z} f(\vec{x}, y, z)$$

Definizione: Operatore di Prodotto Limitato
Sia $f : \mathbb{N}^{k+1} \to \mathbb{N}$. Definiamo $g : \mathbb{N}^{k+1} \to \mathbb{N}$ ponendo:

$$g(\vec{x}, z) = \prod_{y=0}^{z} f(\vec{x}, y)$$

Diciamo che g è ottenuta per **prodotto limitato** da f.

Proposizione: f rp $\implies$ g rp.
(La dimostrazione è la stessa della somma limitata, sostituendo l'operatore di somma s con quello di prodotto p).

Esercizio
Sia $g : \mathbb{N}^2 \to \mathbb{N}$ una funzione rp.

a) Sia $f_1(x, y, z_1, \ldots, z_n) = g(x, y)$. Dimostrare che f_1 è rp:

$$f_1(x, y, z_1, \ldots, z_n) = g(x, y) = \left(g \circ (\mathcal{E}_1^{(n+2)}, \mathcal{E}_2^{(n+2)})\right)(x, y, z_1, \ldots, z_n)$$

b) Sia $f_2(x, y) = g(y, x)$. Dimostrare che f_2 sia rp:

$$f_2(x, y) = \left(g \circ (\mathcal{E}_2^{(2)}, \mathcal{E}_1^{(2)})\right)(x, y)$$

c) Sia $f_3(x) = g(x, x)$. Dimostrare che f_3 sia rp:

$$f_3(x) = \left(g \circ (\mathcal{E}_1^{(1)}, \mathcal{E}_1^{(1)})\right)(x)$$

2.2.2 Relazioni ricorsive primitive

Definizione: Sia $R \subseteq \mathbb{N}^k$. Una funzione $f : \mathbb{N}^k \to \mathbb{N}$ si dice **BUONA per** R quando:

$$\forall\ \vec{x} \in \mathbb{N}^k, \quad f(\vec{x}) = 0 \iff \vec{x} \in R$$

Definizione: $R \subseteq \mathbb{N}^k$ (relazione k-aria) si dice **RICORSIVA PRIMITIVA (rp)** quando:

$$\exists\ f : \mathbb{N}^k \to \mathbb{N} \text{ rp t.c. } \forall\ \vec{x} \in \mathbb{N}^k, \quad f(\vec{x}) = 0 \iff \vec{x} \in R$$

(cioè esiste f rp che è buona per R).

Osservazione: $R \subseteq \mathbb{N}^k$ è rp $\iff$ χ_R (FUNZIONE CARATTERISTICA di R) è rp.

$$\chi_R : \mathbb{N}^k \to \mathbb{N}, \quad \chi_R(\vec{x}) = \begin{cases} 1, & \text{se } \vec{x} \in R \\ 0, & \text{se } \vec{x} \notin R \end{cases}$$

Dimostrazione:

$\implies$) R rp $\implies$ sia $f : \mathbb{N}^k \to \mathbb{N}$ rp BUONA PER R.

$$\chi_R(\vec{x}) = (\overline{sg} \circ f)(\vec{x})$$

Essendo $\overline{sg}$ rp e f rp (per ipotesi), la loro composizione χ_R è rp.

$\impliedby$) χ_R è rp. Osservo che $\overline{sg} \circ \chi_R$ è BUONA PER R.

Proposizione: $R \subseteq \mathbb{N}^k$ rp $\implies$ $R^c = \mathbb{N}^k \setminus R$ rp.
Dimostrazione:
R rp $\implies$ χ_R rp $\implies$ χ_R è funzione BUONA PER R^c ed è rp $\implies$ R^c rp.

Proposizione: $R, S \subseteq \mathbb{N}^k$ rp $\implies$ $R \cup S$ rp.
Dimostrazione:
χ_R, χ_S sono rp.

$$\chi_{R \cup S}(\vec{x}) = sg \circ (\chi_R + \chi_S)(\vec{x})$$

(La somma fa 1 se solo una è vera, o 2 se entrambe vere, poi il sg tronca a 1 se ≥ 2)

$$= (sg \circ (s \circ (\chi_R, \chi_S)))(\vec{x})$$

Essendo ottenuta per composizione di funzioni rp, $\implies R \cup S$ rp.

Proposizione: $R, S \subseteq \mathbb{N}^k$ rp $\implies R \cap S$ rp.
Dimostrazione:
χ_R, χ_S rp.

$$\chi_{R\cap S}(\vec{x}) = \chi_R(\vec{x}) \cdot \chi_S(\vec{x})$$
$$= (p \circ (\chi_R, \chi_S))(\vec{x})$$

Essendo ottenuta per composizione di funzioni rp (prodotto p), $\implies R \cap S$ rp.

Definizione: $f : \mathbb{N}^k \to \mathbb{N}$ si dice **DEFINITA PER CASI** a partire dalle funzioni $f_1, \ldots, f_n$ e dalle relazioni $R_1, \ldots, R_n$ quando:

$$f(\vec{x}) = \begin{cases} f_1(\vec{x}), & \vec{x} \in R_1 \\ f_2(\vec{x}), & \vec{x} \in R_2 \\ \vdots & \vdots \\ f_n(\vec{x}), & \vec{x} \in R_n \end{cases}$$

(Dove $R_1, \ldots, R_n$ è una partizione di $\mathbb{N}^k$). (questa definizione si collega all'idea di "if-then-else" in programmazione)

Proposizione: $f : \mathbb{N}^k \to \mathbb{N}$ definita per casi da $f_1, \ldots, f_n, R_1, \ldots, R_n$.
Se $f_1, \ldots, f_n$ rp e $R_1, \ldots, R_n$ rp $\implies f$ rp.
Dimostrazione:

$$f(\vec{x}) = f_1(\vec{x}) \cdot \chi_{R_1}(\vec{x}) + f_2(\vec{x}) \cdot \chi_{R_2}(\vec{x}) + \cdots + f_n(\vec{x}) \cdot \chi_{R_n}(\vec{x})$$

Poiché somma e prodotto di funzioni rp sono rp, f è rp.

Proposizione:

$$f(\vec{x}) = \begin{cases} f_1(\vec{x}), & \vec{x} \in R_1 \\ \vdots & \vdots \\ f_{n-1}(\vec{x}), & \vec{x} \in R_{n-1} \\ f_n(\vec{x}), & \text{ALTRIMENTI} \end{cases}$$

Se $f_1, \ldots, f_n$ rp e $R_1, \ldots, R_{n-1}$ rp $\implies f$ rp.
Dimostrazione:
Sappiamo che $R_n = \mathbb{N}^k \setminus (R_1 \cup \cdots \cup R_{n-1})$ (che corrisponde all' "altrimenti"). Essendo ottenuta tramite unione e complemento di insiemi rp, R_n è rp $\implies f$ rp.

Esempio
Sia $R \subseteq \mathbb{N}^2$ definita ponendo $(x, y) \in R$ quando $x \leq y$.

$$\chi_R(x, y) = \begin{cases} 1, & x \leq y \\ 0, & x > y \end{cases}$$

$$\chi_R(x, y) = sg(y \dot{-} x + \delta(x, y))$$
$$= \left(sg \circ (s \circ (\dot{-} \circ (\mathcal{E}_2^{(2)}, \mathcal{E}_1^{(2)}), \delta))\right)(x, y)$$

Definizione: Sia $R \subseteq \mathbb{N}^{k+1}$. $f : \mathbb{N}^k \to \mathbb{N}$ è definita per **MINIMIZZAZIONE** da R quando:

$$f(\vec{x}) = \begin{cases} \min\{z \in \mathbb{N} \mid (\vec{x}, z) \in R\}, & \text{SE TALE INSIEME NON È VUOTO} \\ 0, & \text{ALTRIMENTI} \end{cases}$$

Definizione: Sia $R \subseteq \mathbb{N}^{k+1}$. $f : \mathbb{N}^{k+1} \to \mathbb{N}$ è definita per **MINIMIZZAZIONE LIMITATA** da R quando:

$$f(\vec{x}, y) = \begin{cases} \min\{z \leq y \mid (\vec{x}, z) \in R\}, & \text{SE L'INSIEME NON È VUOTO} \\ 0, & \text{ALTRIMENTI} \end{cases}$$

Proposizione: R decidibile $\implies$ f ottenuta per minimizzazione limitata da R è computabile.

Dimostrazione:

Sia $\mathcal{M}$ algoritmo di decisione per R. Costruiamo un algoritmo di calcolo per f:
Su input $(\vec{x}, y) \in \mathbb{N}^{k+1}$:

- Ciclo FOR: per z da 0 a y:
  - Decido se $(\vec{x}, z) \in R$ (usando $\mathcal{M}$)
  - Se sì, restituisco z
- Se il ciclo finisce senza aver trovato niente, restituisco 0.

Proposizione: R rp $\implies$ f ottenuta per minimizzazione limitata da R è rp.

Dimostrazione:

Vogliamo trovare uno schema di Ricorsione Primitiva con f:

$$\begin{cases} f(\vec{x}, 0) = g(\vec{x}) \\ f(\vec{x}, y+1) = \varphi(\vec{x}, y, f(\vec{x}, y)) \end{cases}$$

- $f(\vec{x}, 0) = 0 = C_0^{(k)}(\vec{x})$ (che è rp)
- $f(\vec{x}, y+1) = \begin{cases} f(\vec{x}, y), & \text{SE } \exists\ z,\ 0 \leq z \leq y \text{ t.c. } (\vec{x}, z) \in R \\ y+1, & \text{SE NON SONO NEL CASO PRECEDENTE E } (\vec{x}, y+1) \in R \\ 0, & \text{ALTRIMENTI} \end{cases}$

Dunque definiamo φ:

$$\varphi(\vec{x}, y, t) = \begin{cases} t = \mathcal{E}_{k+2}^{(k+2)}(\vec{x}, y, t) & \text{SE } (\vec{x}, y, t) \in R_1 \\ y+1 = (S \circ \mathcal{E}_{k+1}^{(k+2)})(\vec{x}, y, t) & \text{SE } (\vec{x}, y, t) \in R_2 \\ 0 = C_0^{(k+2)}(\vec{x}, y, t) & \text{ALTRIMENTI} \end{cases}$$

Dove:

$$R_1 = \{(\vec{x}, y, t) \mid \exists\ z,\ 0 \leq z \leq y : (\vec{x}, z) \in R\}$$
$$R_2 = \{(\vec{x}, y, t) \mid (\vec{x}, y, t) \notin R_1 \text{ e } (\vec{x}, y+1) \in R\}$$

Per costruzione, $f(\vec{x}, y+1) = \varphi(\vec{x}, y, f(\vec{x}, y))$. Resta da mostrare che φ è rp. Ricordiamo che, per ipotesi, R è rp, dunque χ_R è rp.

Analizziamo R_1:

$$\chi_{R_1}(\vec{x}, y, t) = sg\left(\underbrace{\sum_{z=0}^{y} \chi_R(\vec{x}, z)}_{\text{operatore di somma limitata applicato a } \chi_R \text{ rp } \implies \text{ rp}}\right)$$

Essendo sg e la somma limitata entrambe operazioni che mantengono la ricorsività primitiva, χ_{R_1} è rp.

Sia $T = \{(\vec{x}, y, t) \in \mathbb{N}^{k+2} \mid (\vec{x}, y+1) \in R\}$.

$$\chi_T(\vec{x}, y, t) = \chi_R(\vec{x}, y+1) = \left(\chi_R \circ (\mathcal{E}_1^{(k+2)}, \ldots, \mathcal{E}_k^{(k+2)}, S \circ \mathcal{E}_{k+1}^{(k+2)})\right)(\vec{x}, y, t)$$

Da cui χ_T è rp.

Infine per R_2:

$$R_2 = R_1^c \cap T$$

Poiché R_1, e di conseguenza R_1^c, sono rp, e T è rp, anche R_2 è rp.

2.2.3 Funzione di Ackermann e funzioni μ-ricorsive

Proposizione: l'insieme delle funzioni rp è **enumerabile**. (si può scrivere un algoritmo che produce una dopo l'altra tutte le funzioni rp).
Dimostrazione: f funzione rp
cioè ha una drp: $f_1, f_2, \ldots, f_n = f$
Posso codificare f usando le funzioni iniziali tramite cui f è costruita usando composizione e RP.

$$\Sigma = \{C, 0, 1, \ldots, 9, S, \mathcal{E}, \text{ ',' }, \circ, (,), R, \langle, \rangle\}$$

(Nota: con C e le cifre codifichiamo una delle funzioni costanti; la virgola serve per separare i parametri; $\circ$ indica la composizione; $\langle\rangle$ si usano per rappresentare l'inizio e la fine della codifica di una funzione)

Regole di codifica:

- $C_0^{(k)} \rightsquigarrow Ck$ (es. $C_0^{(27)} \rightsquigarrow C27$)
- $S \rightsquigarrow S$
- $\mathcal{E}_j^{(k)} \rightsquigarrow \mathcal{E}k, j$ (es. $\mathcal{E}_7^{(12)} \rightsquigarrow \mathcal{E}12, 7$)
- $f \circ (g_1, \ldots, g_n) \rightsquigarrow \langle\langle f\rangle \circ (\langle g_1\rangle, \ldots, \langle g_n\rangle)\rangle$
- f ottenuta per RP da g e φ:
  $f = R(g, \varphi) \rightsquigarrow R(\langle g\rangle, \langle\varphi\rangle)$

Es:

$$S = R(\mathcal{E}_1^{(1)}, S \circ \mathcal{E}_3^{(3)}) \rightsquigarrow \langle R(\langle\mathcal{E}1, 1\rangle, \langle\langle S\rangle \circ (\langle\mathcal{E}3, 3\rangle)\rangle)\rangle$$

Proposizione: $\exists$ una funzione computabile unaria che non è rp.
Dimostrazione:
Per assurdo, supponiamo che ogni funzione computabile unaria sia rp.
Per la proposizione precedente (sulla codifica), l'insieme delle funzioni rp unarie è **enumerabile**.
Cioè $\exists$ un algoritmo di enumerazione che produce la sequenza:

$f_0, f_1, f_2, \ldots, f_x, \ldots$ (lista delle funzioni rp unarie prodotte dall'algoritmo di enumerazione)

Definisco la funzione successore diagonale: $g(x) = f_x(x) + 1$ dove $+ 1$ indica la funzione successore.
Osservazione:

- g è computabile perché f_x e S sono computabili.
- $\exists\ u$ t.c. $g = f_u$ (segue dall'ipotesi iniziale che ogni funzione computabile unaria è rp, e quindi è presente nella lista di enumerazione).

Quanto fa $g(u)$? Dalla definizione di g: $g(u) = f_u(u) + 1$
Ma sapendo che $g = f_u$, si ha: $g(u) = f_u(u)$

$$\Longrightarrow \textbf{ ! ASSURDO ! }$$

FUNZIONE DI ACKERMANN
$A : \mathbb{N}^2 \to \mathbb{N}$

$$\begin{cases} A(0, y) = y + 1 \\ A(x + 1, 0) = A(x, 1) \\ A(x + 1, y + 1) = A(x, A(x + 1, y)) \end{cases}$$

(Dimostra che non tutte le funzioni calcolabili sono rp, e che esistono funzioni che crescono più velocemente di qualsiasi funzione rp).

Esempi:

$$\begin{aligned} A(1, 1) &= A(0, A(1, 0)) \\ &= 1 + A(1, 0) \\ &= 1 + A(0, 1) \\ &= 1 + 2 = 3 \end{aligned}$$

$$
\begin{aligned}
A(2,4) &= A(1,A(2,3)) = 11 \\
A(2,3) &= A(1,A(2,2)) \\
A(2,2) &= A(1,A(2,1)) = A(1,5) = A(0,A(1,4)) \\
A(2,1) &= A(1,A(2,0)) = A(1,A(1,1)) = A(1,3) \\
&= A(0,A(1,2)) \\
&= 1 + A(1,2) \\
&= 1 + A(0,A(1,1)) \\
&= 1 + 4 = 5
\end{aligned}
$$

È una funzione che cresce molto per anche aumenti minimi dei parametri.

- $A(0,y) = y+1$
- $A(1,y) = y+2$
  Dimostrazione per induzione su y:
  - Base: $A(1,0) = A(0,1) = 2 = 0+2$
  - Passo: Supponiamo che $A(1,y) = y+2$.
  $A(1,y+1) = A(0,A(1,y)) = 1 + A(1,y) = 1+y+2 = y+3 = (y+1)+2$
- $A(2,y) = 2y+3$
- $A(3,y) = 2^{y+3} - 3$
- $A(4,y) = \underbrace{2^{2^{\cdot^{\cdot^{\cdot^{2^{16}}}}}}}_{y \text{ volte}} - 3$

PROPRIETÀ DI A

1. $A(x,y) \geq y+1$
2. $A(x,y) < A(x,y+1)$
   Dimostrazione per induzione su x:
   - $x = 0$: $A(0,y) = y+1 < y+2 = A(0,y+1)$
   - Supponiamo che $A(x,y) < A(x,y+1)$
     $A(x+1,y) \underset{\text{per Prop 1}}{\leq} A(x+1,y)+1 \leq A(x,A(x+1,y)) = A(x+1,y+1)$
3. $A(x,y+1) \leq A(x+1,y)$
4. $A(x,y) < A(x+1,y)$
5. $A(x_1,y) + A(x_2,y) < A(\max(x_1,x_2)+4,y)$ (qui la cosa importante è notare come si passi due parametri x_1 e x_2 a un unico parametro $\max(x_1,x_2)+4$)
6. $A(x,y) + y \leq A(x+4,y)$

Proposizione: $\forall\ g : \mathbb{N}^k \to \mathbb{N}$ rp, $\exists\ c \in \mathbb{N}$:

$$\forall\ \vec{x} \in \mathbb{N}^k, \quad g(\vec{x}) < A\left(c, \sum_{i=1}^{k} x_i\right)$$

Dimostrazione: Per induzione strutturale sulla costruzione di g

- Funzioni iniziali:

- $C_0^{(k)}(\vec{x}) = 0 < 1 + \sum_{i=1}^k x_i = A\left(\overset{c=0}{0}, \sum_{i=1}^k x_i\right)$
- $S(x) = x + 1 < x + 2 = A(0, x+1) \overset{(3)}{\leq} A\left(\underset{(c=1)}{1}, x\right)$
- $\mathcal{E}_j^{(k)}(\vec{x}) = x_j < 1 + \sum_{i=1}^k x_i = A\left(\underset{c=0}{0}, \sum_{i=1}^k x_i\right)$

$\implies$ La proposizione vale per le funzioni iniziali.

- **Composizione:** $g(\vec{x}) = h \circ (f_1(\vec{x}), \dots, f_n(\vec{x}))$
  Supponiamo che:
  - $h(x_1, \dots, x_n) < A\left(D, \sum_{i=1}^n x_i\right) \quad (\exists\ D \in \mathbb{N})$
  - $\forall\ j = 1 \dots n, \quad f_j(\vec{x}) < A\left(C_j, \sum_{i=1}^k x_i\right) \quad (\exists\ C_j \in \mathbb{N})$

  Calcoliamo:

$$
\begin{aligned}
g(\vec{x}) = h(f_1(\vec{x}), \dots, f_n(\vec{x})) &< A\left(D, \sum_{i=1}^n f_i(\vec{x})\right) \\
&< A\left(D, \sum_{i=1}^n A\left(C_j, \sum_{i=1}^k x_i\right)\right) \\
&\underset{\text{iterando la (5)}}{\leq} A\left(D, A\left(\widetilde{C}, \sum_{i=1}^k x_i\right)\right) \quad \text{dove } \widetilde{C} = \max_i(C_i) + 4 \\
&\leq A\left(E, A\left(E+1, \sum_{i=1}^k x_i\right)\right) \\
&= A\left(E+1, \sum_{i=1}^k x_i + 1\right) \\
&\leq A\left(E+2, \sum_{i=1}^k x_i\right)
\end{aligned}
$$

Proposizione: A non è rp.
Dimostrazione:
Supponiamo per assurdo che A sia rp.
Allora la funzione $B(x) = A(x, x)$ è rp
(poiché $B(x) = (A \circ (\mathcal{E}_1^{(1)}, \mathcal{E}_1^{(1)}))(x)$)
Dunque per B deve valere la proposizione precedente:

$$\implies \forall\ x \in \mathbb{N}, \exists\ c \in \mathbb{N} \text{ t.c. } B(x) < A(c, x)$$

Calcolo $B(c)$:

$$A(c, c) = B(c) < A(c, c) \quad \textbf{! ASSURDO !}$$

2.2.4 Funzioni μ-ricorsive

Aggiungiamo un costrutto per estendere la classe delle funzioni rp, in modo da riuscire a catturare anche A e altre funzioni che crescono più velocemente di qualsiasi funzione rp.

Definizione: $g : \mathbb{N}^{k+1} \to \mathbb{N}$ si dice **REGOLARE** quando:

$$\forall\ \vec{x} \in \mathbb{N}^k,\ \exists\ y \in \mathbb{N} : g(\vec{x}, y) = 0$$

Definizione: $f : \mathbb{N}^k \to \mathbb{N}$ si dice **OTTENUTA PER MINIMIZZAZIONE** da $g : \mathbb{N}^{k+1} \to \mathbb{N}$ regolare quando:

$$f(\vec{x}) = \min\{y \in \mathbb{N} \mid g(\vec{x}, y) = 0\}$$

Definizione: UNA **DERIVAZIONE μ-RICORSIVA** È UNA SEQUENZA di funzioni $f_1, f_2, \ldots, f_n$ tale che $\forall\ i = 1, \ldots, n$:

- f_i è FUNZIONE INIZIALE, oppure
- f_i è OTTENUTA PER COMPOSIZIONE DA FUNZIONI PRECEDENTI, oppure
- f_i è OTTENUTA PER RP DA 2 FUNZIONI PRECEDENTI, oppure
- f_i è OTTENUTA DA f_j REGOLARE PER MINIMIZZAZIONE (con $j < i$, ovvero f_j è precedente a f_i).

Definizione: UNA FUNZIONE SI DICE **μ-RICORSIVA** QUANDO COMPARE IN CODA A UNA DERIVAZIONE μ-RICORSIVA.

Proposizione: g computabile $\implies$ f computabile *(riferito a queste g ed f)*
Dimostrazione:
Sia $\mathcal{G}$ algoritmo di calcolo per g.
Algoritmo di calcolo per f:

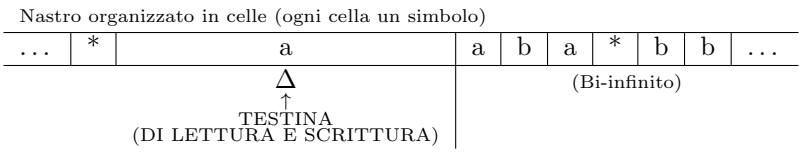
- Data $\vec{x} \in \mathbb{N}^k$
- mentre $y \geq 0$:
- $\left.\begin{array}{l}\text{Calcolo } g(\vec{x}, y) \text{ (usando } \mathcal{G}) \\ \text{Se ottengo 0, restituisco } y\end{array}\right\}$ *(Ciclo While)*

Ci sono indizi che ci fanno pensare che è una classe robusta (più della rp):

- La funzione di Ackermann è μ-ricorsiva.
- Non conosciamo (ad oggi) alcun controesempio.
- Ci sono anche altre proposte di definizioni (ed è quindi molto improbabile che tutte queste definizioni, che sono diverse tra loro, siano tutte sbagliate).

Tesi di Church (per le funzioni μ-ricorsive)
La classe delle funzioni computabili coincide con la classe delle funzioni μ-ricorsive.
(Tesi e non Teorema perché non c'è dimostrazione matematica formale dell'idea di "calcolabile")

2.3 Macchine di Turing



Ogni cella è predisposta per accogliere un simbolo.

$\Sigma \cup \{*\}$ Alfabeto.
Ad ogni istante la macchina, in base a quello che legge e allo stato in cui si trova, può fare una di queste tre operazioni:

- Sovrascrivere (aggiornare) il simbolo
- Spostarsi (a destra o a sinistra)
- Aggiornare lo stato

Q insieme degli stati $\{q_0, q_1, q_2, \dots\}$

Transizione (singolo passo di calcolo/computazione):

$$(\underbrace{q}_{\text{stato}}, \underbrace{X}_{\text{simbolo}}, \underbrace{\alpha}_{\text{azione}}, \underbrace{\tilde{q}}_{\text{stato di arrivo}}) \in Q \times \Sigma \times (\Sigma \cup \{D, S\}) \times Q$$

Dove $\alpha \in \Sigma \cup \{D, S\}$ indica:

- D, S: passo a destra, passo a sinistra
- Se $\alpha \in \Sigma$: scrivo sul nastro senza spostamento

Configurazione: È la coppia (q, X), dove:

- q è lo stato in cui si trova la macchina
- X è la stringa che rappresenta il contenuto del nastro

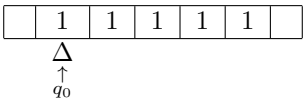
Quartupla funzionale nei primi 2 argomenti: Fissati i primi 2 argomenti (lo stato corrente e il simbolo letto), esiste al più una transizione. Questo garantisce che la macchina sia *deterministica*.

Definizione: Una **Macchina di Turing** (MdT) è una lista di transizioni (cioè una lista di quadruple funzionali nei primi 2 argomenti).

Esempi
1)

$q_0 \ * \ D \ q_1$

$q_0 \ 1 \ D \ q_1$



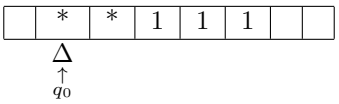
Questa macchina fa solo un passo a destra e si ferma.

2)

$q_0 \ * \ D \ q_0$

$q_0 \ 1 \ D \ q_1$

$q_1 \ 1 \ D \ q_1$



Legge gli asterischi spostandosi a destra; quando trova un 1*, passa allo stato* q_1 *e continua a scorrere gli* 1 *verso destra.*

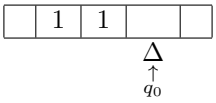
3)

$q_0 \ * \ 1 \ q_0$

$q_0 \ 1 \ D \ q_1$

$q_1 \ * \ 1 \ q_1$

$q_1 \ 1 \ 1 \ q_0$



Mette tutti 1 *e non termina mai.*
(Nota: si può scrivere anche come $q_0 \ * \ 1 \ q_0$ *seguito da* $q_0 \ 1 \ D \ q_0$*, poiché per fare la stessa cosa ci possono essere più Macchine di Turing.*

4) Funzione Successore

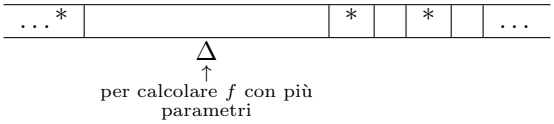
$q_0 \ * \ D \ q_1$

$q_1 \ 1 \ D \ q_1$

$q_1 \ * \ 1 \ q_2$

$q_2 \ 1 \ S \ q_2$

Definizione: Una MdT M **calcola una funzione** f quando, fatta partire sulla cella vuota all'immediata sinistra dell'input, termina sulla cella vuota all'immediata sinistra dell'output.



Cioè si inserisce degli * per delimitare i parametri.

5) Funzione Somma

$q_0 \ * \ D \ q_1$

$q_1 \ 1 \ D \ q_1$

$q_1 \ * \ 1 \ q_2$

$q_2 \ 1 \ D \ q_2$

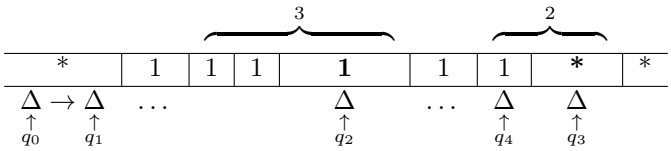
$q_2 \ * \ S \ q_3$

$q_3 \ * \ S \ q_4$

$q_4 \ 1 \ * \ q_4$

$q_4 \ * \ S \ q_5$

$q_5 \ 1 \ S \ q_5$



(Nota: La macchina scorre a destra trasformando l'asterisco centrale in un 1*, unendo così i due blocchi. Arrivata in fondo, cancella gli ultimi due* 1 *sostituendoli con degli* $*$ *per compensare l'eccesso della codifica unaria e mantenere il risultato corretto della somma. Infine, torna all'indietro fino all'asterisco iniziale per fermarsi).*

Funzione τ-ricorsiva: Una funzione $f : \mathbb{N}^k \to \mathbb{N}$ si dice τ-ricorsiva quando $\exists$ M MdT che calcola f.

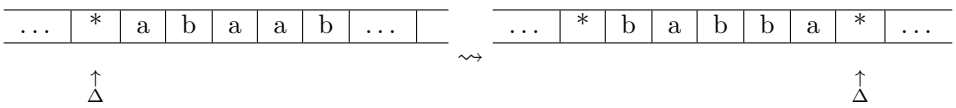
Tesi di Church (per funzioni τ-ricorsive): La classe delle funzioni computabili coincide con la classe delle funzioni τ-ricorsive.

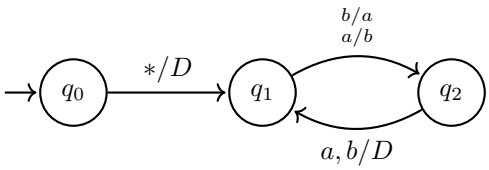
Esercizio
Siano $f : \mathbb{N} \to \mathbb{N}$ rp, $R \subseteq \mathbb{N}^k$ rp (quindi χ_R è rp).
Definisco $S \subseteq \mathbb{N}^k$ ponendo $\vec{x} \in S$ usando $(f(x_1), \dots, f(x_k)) \in R$.
Dimostrare che S è rp.
Dimostrazione:

$$\chi_S(\vec{x}) = \chi_R \circ (f \circ \mathcal{E}_1^{(k)}, \dots, f \circ \mathcal{E}_k^{(k)})(\vec{x})$$

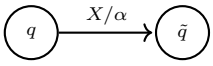
Essendo S definita tramite la composizione della funzione caratteristica di R (che è rp per ipotesi) con una serie di composizioni di f (rp per ipotesi) e le proiezioni (rp in quanto funzioni iniziali), la funzione χ_S è ottenuta per composizione di funzioni rp, ed è quindi essa stessa rp. Di conseguenza, l'insieme S è rp.

6) MDT che, data una stringa su $\{a, b\}$, scambia le a con le b





Rappresentazione grafica di una MDT:



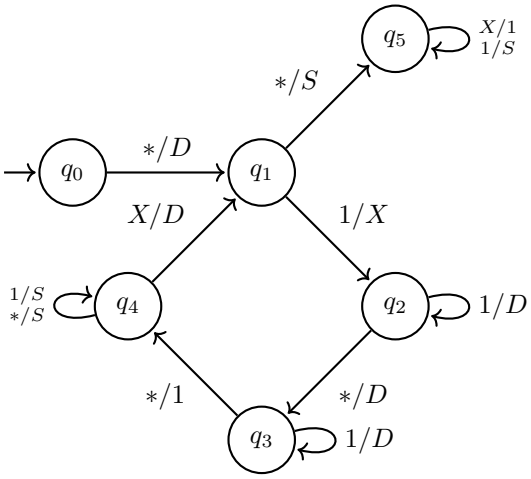
7) MDT che scrive la copia di una stringa unaria

...	*	1	1	1	1	*	...
	$\uparrow$ q_0						

$\rightsquigarrow$

...	*	1	1	1	1	*	1	1	1	1	*	...
	$\uparrow$ q_5											

Nota: si utilizza un simbolo aggiuntivo (di lavoro) X per indicare la copia di un 1.



(Nota: la macchina legge un 1*, lo sostituisce con una X e scorre a destra. Salta l'asterisco separatore e gli eventuali* 1 *già copiati, fino a trovare la prima cella vuota ($*$) dove scrive un* 1*. Poi torna indietro verso sinistra fino a ritrovare la X per ripetere il processo. Quando non ci sono più* 1 *da copiare, la macchina entra nella fase di ripristino: scorre verso sinistra sostituendo ogni X con un* 1 *per ricreare l'input originale, finché non incontra l'asterisco iniziale, dove si ferma definitivamente).*

2.3.1 Macchine di Turing come accettatori di linguaggi

Sia Q l'insieme degli stati di una MDT e $F \subseteq Q$ l'insieme degli stati finali.

Definizione: Una macchina M accetta la stringa w per stati finali quando l'esecuzione di M su input w termina in uno stato finale.

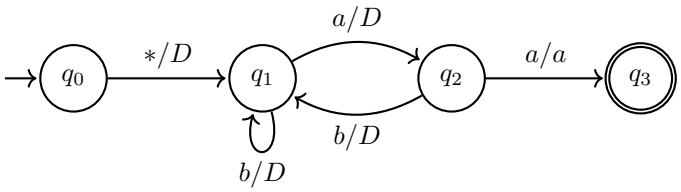
Definizione: Un linguaggio $L \subseteq \Sigma^*$ si dice accettato per stati finali da una MDT M quando: $w \in L \iff M$ accetta w per stati finali. In questo caso si scrive $L = L(M)$.

Definizione: Se L è tale che $\exists M$ MDT per cui $L = L(M)$, l'insieme L si dice **ricorsivamente enumerabile**.

Osservazione: Se M è una MDT che termina su ogni input, allora $L(M)$ si dice **ricorsivo**.

Via Tesi di Church:
ricorsivamente enumerabile $\iff$ semidecidibile
ricorsivo $\iff$ decidibile

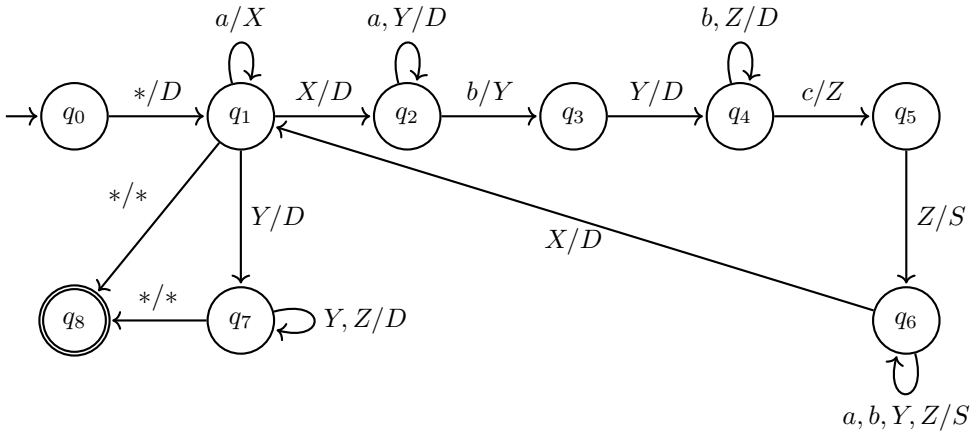
Esempio 1) MDT che accetta il linguaggio $(a|b)^*aa(a|b)^*$



$Q = \{q_0, q_1, q_2, q_3\}, \quad F = \{q_3\}$

(Nota: la macchina, dopo aver trovato due a consecutive, entra nello stato finale q_3 e si ferma, accettando così la stringa a prescindere da quello che c'è dopo le due a).

Esempio 2) MDT che accetta il linguaggio $\{a^i b^i c^i \mid i \geq 0\} \subseteq \{a, b, c\}^*$



Nota esplicativa sul funzionamento:
La macchina non potendo "contare" la lunghezza i, procede accoppiando i simboli a tre a tre tramite passate successive, utilizzando i simboli ausiliari X, Y, Z come marcatori.

- ***Ciclo di marcatura*** *($q_1 \to q_6$)**:*** *Partendo da sinistra, la macchina legge la prima a disponibile e la marca scrivendo X. Scorre a destra saltando le a e le Y (q_2), trova la prima b e la marca con Y (q_3). Scorre ancora a destra saltando le b e le Z (q_4), trova la prima c e la marca con Z (q_5). A questo punto inverte la marcia (q_6) e torna tutta a sinistra fino a sbattere contro l'ultima X scritta, per poi fare un passo a destra e ricominciare un nuovo ciclo.*
- ***Fase di verifica*** *(q_7)**:*** *Quando in q_1 la macchina legge una Y invece di una a, significa che le a sono finite. Entra quindi in fase di verifica (q_7): scorre tutto il nastro a destra assicurandosi che siano rimaste solo Y e Z (nessuna b o c in eccesso).*
- ***Accettazione*** *(q_8)**:*** *Se arriva in fondo leggendo solo marcatori e incontra l'asterisco finale ($*$), la stringa è perfettamente bilanciata e viene accettata per arresto nello stato finale q_8. La transizione $q_1 \xrightarrow{*/*} q_8$ serve invece per accettare la stringa vuota (caso $i = 0$).*

Definizione: Una MDT M accetta w **per arresto** quando M, eseguita su w, termina. (in qualsiasi stato).

Definizione: L'insieme delle stringhe accettate da M per arresto è il linguaggio accettato da M per arresto.

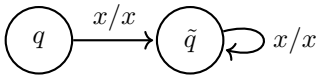
Proposizione: Sia L un linguaggio. $\exists M$ MDT che accetti L per stati finali $\iff$ $\exists N$ MDT che accetta L per arresto.
Dimostrazione:

$\Longleftarrow$) Sia N una MDT che accetta L per arresto. La MDT M ottenuta da N designando ogni stato come stato finale è una MDT che accetta L per stati finali.

$\Longrightarrow$) Sia M una MDT che accetta L per stati finali. Su input w:

- M termina in uno stato finale (selezionando Q = stati di M, F = stati finali di M)
- M termina in uno stato non finale
- M non termina

Costruiamo la MDT N in questo modo: N ha le stesse transizioni di M, più le seguenti: $\forall q \in Q \setminus F$, $\forall x \in \Sigma$ tali che non ci sono transizioni di M con configurazione (q, x), aggiungiamo le transizioni $q \xrightarrow{x/x} \tilde{q}$ e $\tilde{q} \xrightarrow{x/x} \tilde{q}$ (dove $\tilde{q}$ è un nuovo stato in cui la macchina "cicla" all'infinito senza mai fermarsi).



Esercizi

1) L è accettato per un unico stato finale quando $\exists M$ MDT con un solo stato finale che accetta L. Dimostrare che L è accettato per stati finali $\iff$ è accettato per un unico stato finale.

Dimostrazione:

$\Longleftarrow$) Ovvio.

$\Longrightarrow$) Sia M una MDT che accetta L per stati finali. Costruiamo N aggiungendo un nuovo stato $\tilde{q}$, che M raggiungerà ogni volta che termina in uno stato finale (inoltre $\tilde{q}$ sarà l'unico stato finale di N).

2) M accetta w **per ingresso** quando, durante l'esecuzione di M su w, la MDT entra in uno stato finale. Un linguaggio L è accettato da M per ingresso quando $\exists M$ MDT che accetta tutte e sole le stringhe di L per ingresso. Dimostrare che L è accettato per stati finali $\iff$ L è accettato per ingresso.

Dimostrazione:

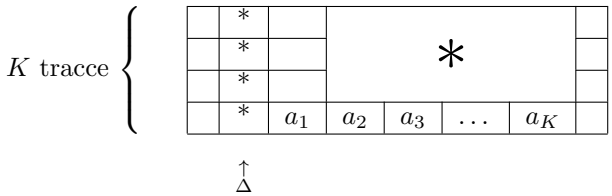
$\Longrightarrow$) Sia M una MDT che accetta L per stati finali. M' che accetta per ingresso si costruisce a partire da M aggiungendo un nuovo stato $\tilde{q}$ (che sarà l'ultimo stato finale) e transizioni che portano in $\tilde{q}$ da ogni stato finale in corrispondenza a caratteri per cui non ci sono transizioni uscenti in M.

$\Longleftarrow$) Sia M una MDT che accetta L per ingresso. M' MDT che accetta L per stati finali si ottiene da M eliminando tutte le transizioni uscenti dagli stati finali.

2.3.2 Varianti di Macchine di Turing

MDT multitraccia: Ha Σ alfabeto, Q insieme degli stati e una lista di transizioni della forma:

$$(q, (x_1, \ldots, x_k), \alpha, \tilde{q}) \in Q \times \Sigma^k \times (\Sigma^k \cup \{D, S\}) \times Q$$



Cioè la testina legge un vettore di k simboli.

Proposizione: Fissato $K \in \mathbb{N}$, L è accettato da una MDT a una traccia $\iff$ L è accettato da una MDT a K tracce.

Dimostrazione:

$\Longrightarrow$) Ovvio (posso simulare una MDT con una traccia, usando una MDT con il contenuto di K tracce vuote, tranne la prima).

$\Longleftarrow$) Sia M una MDT a K tracce che accetta L. Posso ottenere una MDT M' che accetta L a una traccia semplicemente sostituendo l'alfabeto Σ di M con l'alfabeto esteso Σ^K (le cui lettere sono di fatto K-uple), ed eseguendo le stesse transizioni.

MDT limitata a sinistra:

*	a_1	...	a_n	...

$\uparrow$
Δ

Possiamo usare una MDT limitata a sinistra per accettare stringhe nello stesso modo di quelle classiche, con l'accortezza che un'operazione di spostamento a sinistra a partire dalla prima cella causa il rifiuto della stringa.

Proposizione: L accettato da una MDT classica $\iff$ L accettato da una MDT limitata a sinistra.
Idea di Dimostrazione:

$\Longleftarrow$) **Da Limitata a Classica:** Sia M una MDT limitata a sinistra che accetta L. Vogliamo simularla con una MDT classica (a nastro bi-infinito). La macchina classica farà le stesse identiche cose, ma posizionerà un simbolo speciale (ad esempio #) nella cella di partenza (indice 0). Se durante la computazione la macchina legge questo simbolo e riceve l'istruzione di spostarsi a sinistra, saprà che la mossa è "illegale" e rifiuterà la stringa, simulando esattamente il blocco della macchina limitata.

$\Longrightarrow$) **Da Classica a Limitata:** Sia M una MDT classica che accetta L. Immaginiamo di numerare le celle del suo nastro infinito partendo dalla cella di partenza come indice 0 (avremo celle positive a destra $1, 2, 3 \ldots$ e negative a sinistra $-1, -2, -3 \ldots$). Creiamo una MDT limitata a sinistra dotata di **2 tracce**, che funziona come se avessimo "piegato a metà" il nastro originale proprio sullo zero:

	-0	-1	-2	-3		
Traccia 2 (indici negativi) $\rightarrow$						...
Traccia 1 (indici ≥ 0**)** $\rightarrow$	*	a_1	a_2	a_3		...
	0	1	2	3		

La nuova macchina replica il comportamento di M per le celle ≥ 0 operando sulla prima traccia (quella inferiore). Quando la macchina originale cercherebbe di passare a sinistra dello 0 (verso gli indici negativi), la nostra nuova macchina passa a operare sulla seconda traccia (quella superiore) e **inverte gli spostamenti**: poiché il nastro è ripiegato, un passo a "sinistra" della macchina originale diventa un passo a "destra" sulla nostra traccia ripiegata, e viceversa.

NB: MDT standard $\iff$ MDT limitata a sinistra.

MDT multinastro: Opera su K nastri, ciascuno dotato della propria testina di lettura/scrittura. Le K testine possono operare e muoversi in modo totalmente indipendente l'una dall'altra. Una transizione di una MDT multinastro è della forma:

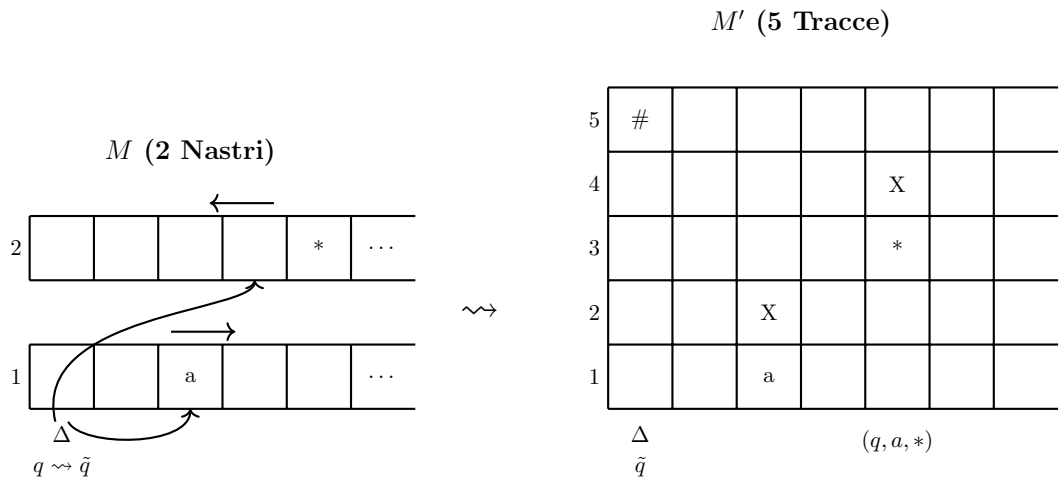
$$(q, (x_1, \ldots, x_k), (\alpha_1, \ldots, \alpha_k), \tilde{q}) \in Q \times \Sigma^K \times (\Sigma \cup \{D, S\})^K \times Q$$

Proposizione: L accettato da una MDT standard $\iff$ L accettato da una MDT multinastro.
Dimostrazione:

$\Longrightarrow$) Ovvio (basta ignorare i nastri in eccesso).

$\Longleftarrow$) Sia M una MDT a K nastri che accetta L. Facciamo vedere che $\exists$ una MDT M' a $2K+1$ tracce che accetta L. Più precisamente facciamo vedere che ogni singola transizione di una computazione di M può essere simulata da un gruppetto di transizioni di M' (usando una traccia per il contenuto di ogni nastro e una traccia ausiliaria per marcare la posizione di ogni singola testina).

Transizioni di M' (Simulazione di una MDT a K nastri con una a $2K+1$ tracce):
Esempio con $K = 2$: La MDT M ha 2 nastri indipendenti, quindi la MDT M' che la simula avrà $2(2) + 1 = 5$ tracce.



Come funziona la simulazione (su M'):

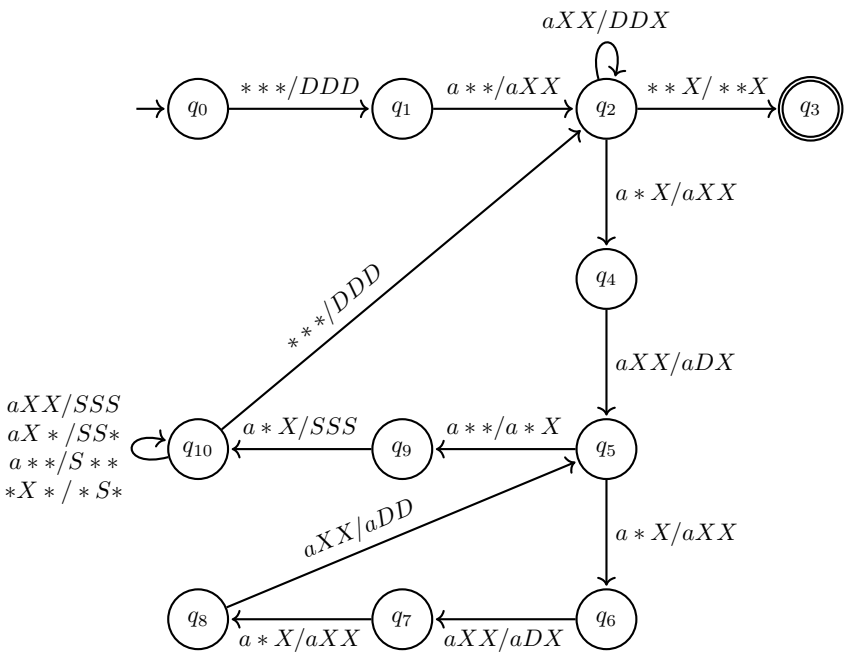
- La prima cosa che fa è inserire un simbolo speciale (#) nell'ultima traccia (la traccia 5) per marcare l'inizio.
- La prima traccia viene usata per il contenuto del primo nastro.
- La terza traccia viene usata per il contenuto del secondo nastro.
- La seconda e la quarta traccia memorizzano la posizione delle rispettive testine (segnando una X nella colonna in cui si troverebbe la testina).
- Per eseguire una transizione, parto sempre con la testina all'inizio (sul #).
- Scorro in avanti, guardo la X di traccia 2 e memorizzo il simbolo associato in traccia 1 (in questo caso a).
- Torno indietro fino al # e faccio la stessa cosa per il secondo nastro (cercando la X in traccia 4 per memorizzare il simbolo in traccia 3).
- Torno di nuovo all'inizio, e poi faccio le scritture e il movimento della X allo stesso modo (sempre andando avanti e indietro a partire dal #).

Nota: Per memorizzare i simboli letti durante questi passaggi, l'idea è utilizzare la memoria interna degli stati. Definiamo gli stati di M' come tuple, ad esempio $(q, a, *)$, che "ricordano" sia lo stato originale q in cui ci si trovava, sia i simboli appena letti.

Esempio: MDT a 3 nastri che decide il linguaggio (su $\{a\}$) $\{a^K \mid K$ è un quadrato perfetto$\}$

Nastro 1 (K):		X	X					
Nastro 2 (K^2):		X	X	X	X			
Nastro 3 (Input):	*	a	a	a	a	a	a	...
	$\uparrow$ Δ							

Logica matematica per l'aggiornamento: $K^2 \rightsquigarrow (K+1)^2 = K^2 + 2K + 1$



2.4 MDT Non Deterministica

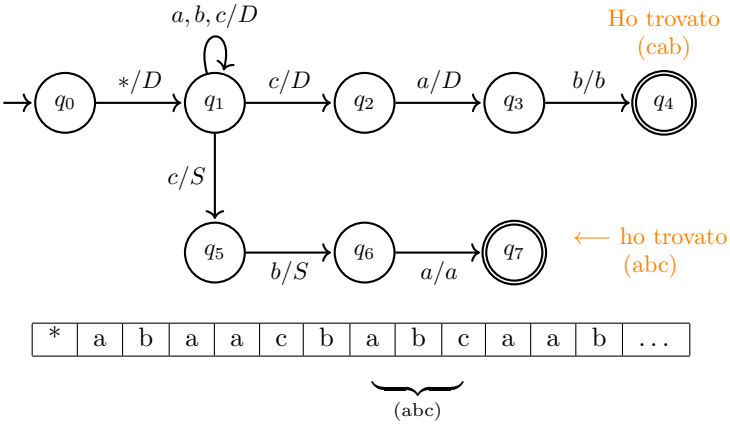
Definizione: Una MDT si dice **non deterministica** quando le transizioni non sono necessariamente funzionali nei primi 2 argomenti. (Ovvero, a parità di stato e simbolo letto, la macchina può avere più scelte su cosa fare).

Osservazione: Le MDT deterministiche (= standard) sono MDT non deterministiche (ovvero, con una sola scelta per ogni configurazione).

Definizione: Data una stringa w e una MDT non deterministica M, diciamo che M accetta w quando **esiste** (almeno) una computazione di M che accetta w.

Esempio:

$$L = \{w \in \{a,b,c\}^* \mid \exists \text{ un'occorrenza di 'c' immediatamente preceduta da } ab \text{ (abc)}$$
$$\text{oppure immediatamente seguita da } ab \text{ (cab)}\}$$



Grado di non determinismo di M: Data M MDT non deterministica, il suo grado di non determinismo δ è il massimo numero di scelte possibili per una singola configurazione:

$$\delta = \max_{\substack{\tilde{q}\in Q \\ \tilde{x}\in\Sigma}} |\{\tilde{q}\tilde{x}\alpha q \mid \alpha \in \Sigma \cup \{D, S\}, q \in Q\}|$$

Dato $q \in Q, x \in \Sigma$ codifichiamo le transizioni di M (MDT non deterministica) aventi qx come primi 2 elementi utilizzando gli interi da 1 a δ, possibilmente codificando la stessa transizione con più etichette.

- $q_1\ c\ D\ q_1\ :\ 1 \qquad q_1\ a\ D\ q_1\ :\ 1, 2, 3$
- $q_1\ c\ D\ q_2\ :\ 2$
- $q_1\ c\ S\ q_5\ :\ 3$

Esempi di possibili computazioni sulla stringa $w = acab$:

1. 1111, termina in uno stato non finale (w *non accettata*)
2. 11211, termina in uno stato finale (w **accettata**)
3. 22322, termina subito (w *non accettata*)

Proposizione: L è accettato da una MDT standard $\iff$ L è accettato da una MDT non deterministica.
Dimostrazione:

$\Longrightarrow$) Sia M una MDT standard che accetta L. Poiché ogni MDT deterministica è (banalmente) anche una MDT non deterministica, L è accettato da una MDT non deterministica.

$\Longleftarrow$) Sia M una MDT non deterministica che accetta L per arresto, con grado di non determinismo di $M = \delta$. Descriviamo una MDT *deterministica* che accetta L per arresto a 3 nastri (per simulare tutte le computazioni parallele di M in modo sistematico):

3	m_1	*	m_2	*	...	Generazione delle computazioni di M ($1 \leq m_i \leq \delta$)
2	*	copio l'input ed eseguo ($m_1, \ldots, m_K$)				Simulazione delle computazioni di M
1						Input

2.5 Funzioni Parziali Computabili

Funzione parziale: Dati A, B insiemi, una funzione parziale da A a B è una funzione da D a B, per qualche dominio $D \subseteq A$. *(Cioè potrebbe essere definita solo su una parte di A, ad es. $f(x) = \frac{10}{x}$, che non è definita per $x = 0$).*

Funzione parziale computabile: Data una funzione parziale f da A a B, si dice computabile quando $\exists M$ algoritmo tale che $\forall x \in A$:

- Se $x \in$ Dominio(f), M eseguita su x restituisce in output $f(x) \longrightarrow x \in$ Dominio(f): f converge su x, si scrive $f(x)\downarrow$
- Se $x \notin$ Dominio(f), M su x non termina $\longrightarrow x \notin$ Dominio(f): f diverge su x, si scrive $f(x)\uparrow$

Lavoriamo su $\mathbb{N}$ in codifica unaria.
Definizione: f funzione parziale da $\mathbb{N}^k \to \mathbb{N}$ si dice **parziale τ-ricorsiva** quando $\exists M$ MDT che, $\forall \vec{x} \in \mathbb{N}^k$:

- Se $f(\vec{x})\downarrow$ allora l'esecuzione di M su $\vec{x}$ termina con $f(\vec{x}) + 1$ "uni" sul nastro.
- Se $f(\vec{x})\uparrow$ allora l'esecuzione di M su $\vec{x}$ non termina su $\vec{x}$.

Tesi di Church per funzioni parziali τ-ricorsive: La classe delle funzioni parziali computabili coincide con la classe delle funzioni parziali τ-ricorsive.

Proposizione: $\forall M$ MDT deterministica, $\forall K \in \mathbb{N}$ $\exists!$ funzione parziale computabile $f : \mathbb{N}^k \to \mathbb{N}$ tale che M calcola f.

Dimostrazione: Definisco f funzione parziale da $\mathbb{N}^k$ in $\mathbb{N}$ come segue:
$\forall \vec{x} \in \mathbb{N}^k$, eseguo M su $\vec{x}$:

- Se M termina, $f(\vec{x})$ è data in unario dal numero di "uni" che si trovano sul nastro al termine dell'esecuzione.
- Se M non termina, $f(\vec{x}) \uparrow$. Questo ci dice che ad ogni M corrisponde (in modo univoco) una funzione f ($M \longleftrightarrow f$).